

Code structure & circuit-to-code map

Two diagrams: the Python package's module-dependency structure, and how each physical device in the MIM signal chain maps to a code class and its ports.

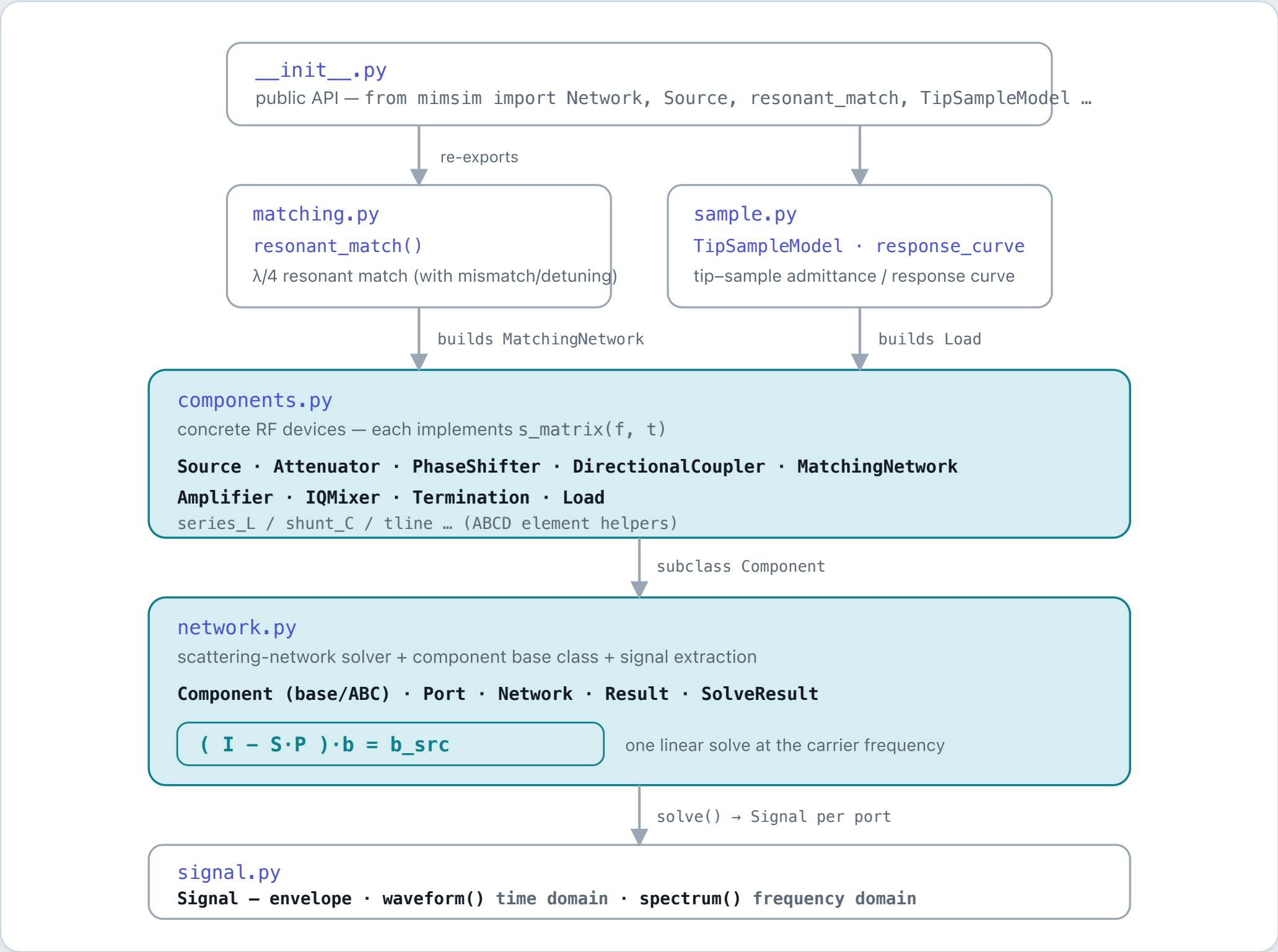
physical RF path / solver

code identifier (class / function)

measured output / probeable quantity

Fig. 1 Code structure (module dependencies)

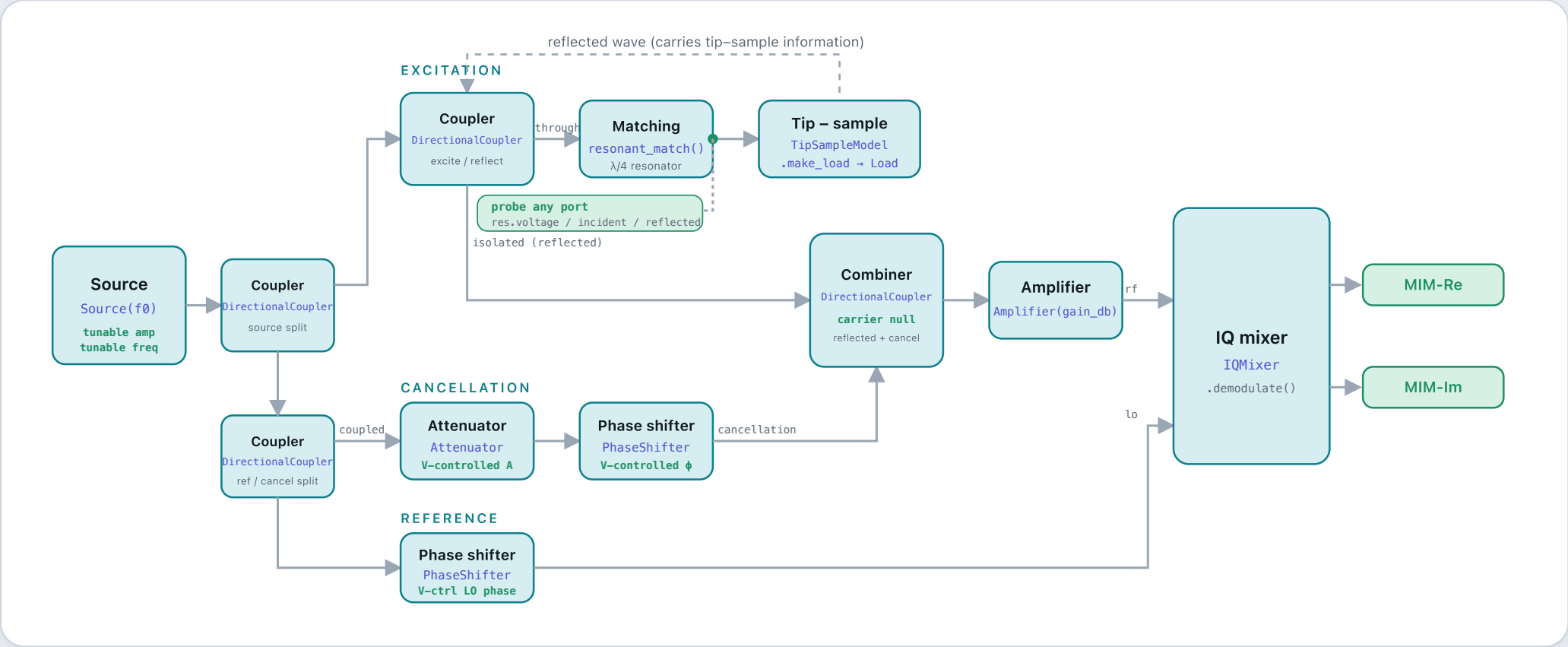
Arrows point in the "depends on / uses" direction. Bottom: signal representation. Middle: the scattering-network solver and component base class. Top: concrete devices and domain models, then the public API.



Dependencies converge downward: the domain layer (matching.py , sample.py) assembles circuits from the component layer; every component class subclasses Component and implements s_matrix(f,t) ; the solver wraps each port wave as a Signal .

Fig. 2 Circuit structure and its code mapping

The MIM reflectometer as three arms tapped from one source (cf. paper Fig. 1a). **Excitation** drives the tip; **cancellation** (voltage-controlled attenuator A + phase shifter φ) synthesizes a wave that **meets the reflected wave at the combiner and nulls the carrier** before amplification; **reference** (phase shifter φ) sets the LO phase for the IQ mixer.



One **Source** feeds three arms. **Excitation:** `DirectionalCoupler` → `resonant_match` → `Load`. The reflection leaves the excitation coupler's `isolated` port and meets the **cancellation** wave — synthesized by the voltage-controlled `Attenuator` + `PhaseShifter` — at the **combiner**, nulling the carrier before `Amplifier` → `IQMixer` RF. The **reference** `PhaseShifter` supplies the LO phase. `demodulate()` → MIM-Re / MIM-Im. Green dot: any port yields a time- or frequency-domain signal after solving.